

Previsione della tossicità molecolare con Graph Isomorphism Networks

Youness Aharram — matricola 7198778

Project work di *Fundamentals of Machine Learning* (3 CFU)

Sommario

In questo project work è stata affrontata la previsione della tossicità di molecole rappresentandole come grafi, in cui i nodi sono gli atomi e gli archi i legami chimici. Ho addestrato una *Graph Isomorphism Network* (GIN) sul dataset Tox21 nella versione `ogbg-moltox21` di Open Graph Benchmark, che contiene 7831 molecole e 12 etichette binarie ciascuna, con il 17% delle etichette assenti e classi fortemente sbilanciate. Come termine di paragone non neurale ho usato una Random Forest addestrata su una rappresentazione della molecola chiamata fingerprint ECFP4, un vettore binario che codifica la presenza o l'assenza di specifiche sottostrutture locali della molecola. Ho poi condotto sette esperimenti controllati su tipo di convoluzione, pooling globale, profondità della rete, criterio di partizione dei dati e riequilibrio delle classi nella loss. Il risultato principale è che la GIN con pooling *max* raggiunge 0,7564 di ROC-AUC sul test contro 0,7325 della baseline, ma perde sull'average precision (0,3632 contro 0,3939).

1 Introduzione e obiettivi

L'idea del progetto nasce dall'interesse di voler comprendere se l'applicazione delle tecniche di **machine learning** a questo tipo di problema possa rappresentare un ambito di approfondimento per il futuro. Il progetto ha come obiettivo principale quello di voler comprendere il funzionamento di una **Graph Isomorphism Network (GIN)** e del **message passing**, imparare a utilizzare **PyTorch Geometric**, in particolare il modo in cui gestisce i batch di grafi disconnessi, e approfondire il ruolo delle scelte architetturali e del **global pooling**. Un ulteriore obiettivo è imparare a valutare correttamente un problema **multi-label** e fortemente sbilanciato, utilizzando metriche appropriate. Il dominio applicativo scelto è quello della **tossicologia computazionale**, in cui le molecole vengono rappresentate come grafi e utilizzate per prevederne la tossicità. Un aspetto particolarmente interessante dei modelli a grafo è che, rispetto agli approcci basati su descrittori costruiti manualmente, permettono alla rete di **apprendere direttamente dai dati quali caratteristiche strutturali della molecola siano rilevanti** per il compito. Infine, il progetto mira a confrontare il comportamento di una **GNN addestrata da zero su poche migliaia di molecole** con quello di un metodo di machine learning tradizionale, una **Random Forest basata su fingerprint ECFP4**, analizzando non solo le prestazioni complessive, ma anche l'influenza delle diverse scelte architetturali e sperimentali sui risultati.

2 Il dataset Tox21

Tox21 (*Toxicology in the 21st Century*) è un programma di screening ad alto rendimento finalizzato a studiare l'attività di numerosi composti attraverso una serie di saggi biologici. In

questo progetto ho utilizzato la versione `ogbg-moltox21` distribuita da **Open Graph Benchmark (OGB)**, che fornisce le molecole già rappresentate come grafi per **PyTorch Geometric**, a partire dalla loro rappresentazione SMILES e attraverso una featurizzazione standard. Un aspetto particolarmente importante per questo lavoro è la disponibilità di uno *scaffold split* ufficiale, che rende la suddivisione tra training, validation e test riproducibile e comune a tutti gli esperimenti condotti sul dataset. Lo *scaffold split* suddivide le molecole in base al loro *scaffold* strutturale, facendo in modo che molecole con strutture di base simili non vengano distribuite contemporaneamente nei diversi sottoinsiemi.

Le principali caratteristiche del dataset sono riportate nella Tabella 1.

Tabella 1: Caratteristiche di `ogbg-moltox21`.

molecole (grafi)	7831
task binari per molecola	12
etichette mancanti	17,05% delle celle della matrice 7831×12
attributi per nodo (atomo)	9 categorici (numero atomico, carica, chiralità, ...)
attributi per arco (legame)	3 categorici (tipo di legame, stereochimica, coniugazione)
atomi per molecola	mediana 16, media 18,6, da 1 a 132
legami per molecola	media 19,3 (38,6 archi diretti)
split scaffold ufficiale	6264 train / 783 validation / 784 test

I 12 task rappresentano due principali tipologie di saggi biologici. I sette task **NR-*** misurano l'interazione con recettori nucleari, tra cui **NR-AR**, relativo al recettore degli androgeni, mentre i cinque task **SR-*** riguardano l'attivazione di specifiche vie di risposta allo stress cellulare, tra cui **SR-MMP**, associato al potenziale di membrana mitocondriale. Per ciascun task, l'etichetta assume valore 1 se il composto è risultato attivo nel relativo saggio e 0 se è risultato inattivo. Un valore NaN, invece, indica che il composto non è stato testato per quel particolare task. Due caratteristiche del dataset risultano particolarmente rilevanti per il resto del progetto: il forte sbilanciamento delle classi e la presenza di un numero significativo di etichette mancanti. Per questo motivo, prima di procedere con l'addestramento ho condotto un'analisi esplorativa dei dati (si può trovare il notebook relativo all'esplorazione del dataset nella repository su github), i cui risultati sono mostrati in Figura 1.

Tox21: 7831 molecole, 12 task binari (17.1% delle etichette assenti)

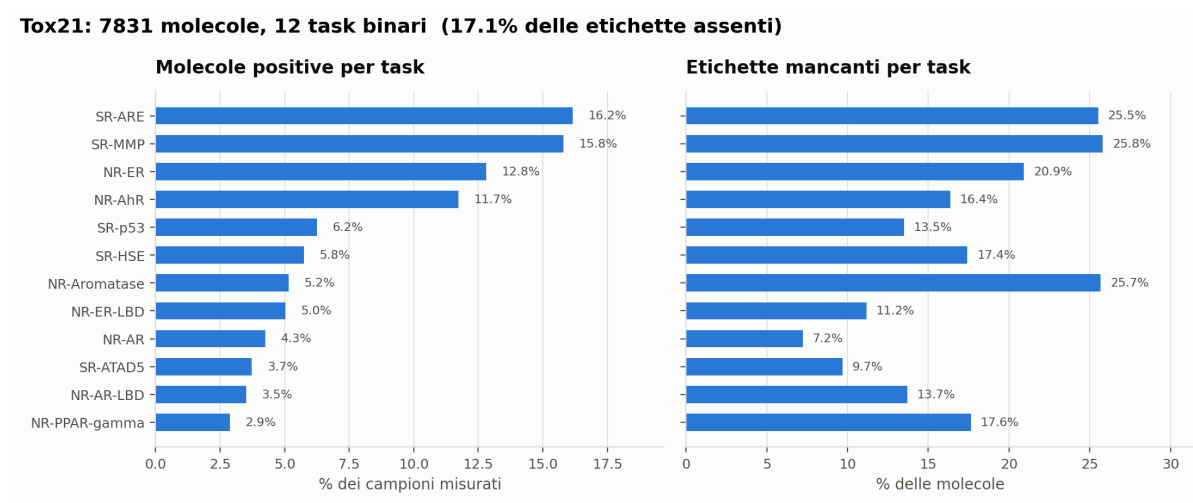


Figura 1: A sinistra, percentuale di molecole positive per ciascun task; a destra, percentuale di etichette mancanti. I task sono ordinati in base alla percentuale di positivi.

3 Preparazione dei dati

3.1 Dai grafi ai batch

Il vantaggio di utilizzare la versione fornita da OGB è che la conversione da SMILES a grafo e la relativa featurizzazione sono già definite e applicate in modo uniforme a tutte le molecole, rendendo la rappresentazione dei dati riproducibile e i risultati più facilmente confrontabili. Gli *SMILES* (*Simplified Molecular Input Line Entry System*) sono una rappresentazione testuale della struttura di una molecola: attraverso una sequenza di caratteri descrivono gli atomi presenti e i legami che li collegano. Gli attributi di nodi e archi sono rappresentati come indici categorici, non come valori numerici continui. Per questo motivo è stato utilizzato l'**AtomEncoder** di OGB, che associa a ciascun attributo un *embedding* apprendibile; gli embedding dei diversi attributi vengono poi sommati per ottenere la rappresentazione iniziale di ogni atomo. Un procedimento analogo viene utilizzato per gli attributi dei legami tramite il **BondEncoder**. Un aspetto importante riguarda la gestione dei *mini-batch* di grafi di dimensione diversa. PyTorch Geometric non utilizza il *padding*: unisce i grafi in un'unica struttura composta da componenti disconnesse e aggiorna gli indici degli archi di conseguenza. Il vettore **batch** indica, per ogni nodo, a quale grafo appartiene. In questo modo il *message passing* rimane confinato all'interno di ciascuna molecola, mentre il *global pooling* utilizza **batch** per raggruppare i nodi e ottenere una rappresentazione separata per ogni molecola.

3.2 Le etichette mancanti

Le etichette mancanti sono state gestite attraverso una *maschera*. In pratica, durante l'addestramento vengono considerate solamente le coppie molecola-task per le quali è disponibile un'etichetta. La loss utilizzata è quindi una *binary cross-entropy* calcolata sui soli valori osservati:

$$\mathcal{L} = \frac{1}{|\mathcal{O}|} \sum_{(i,t) \in \mathcal{O}} \text{BCE}(\hat{y}_{it}, y_{it}), \quad \mathcal{O} = \{(i, t) : y_{it} \neq \text{NaN}\}.$$

La media viene calcolata sul numero effettivo di etichette disponibili, e non sul numero totale di possibili etichette. In questo modo, la presenza di un numero diverso di valori mancanti nei vari batch non altera il valore della loss. Prima di applicare la BCE, i valori **NaN** vengono temporaneamente sostituiti con zero e successivamente esclusi attraverso la maschera. Questo è necessario perché, in aritmetica floating-point, operazioni che coinvolgono un **NaN** possono produrre a loro volta **NaN**, anche quando il risultato viene successivamente moltiplicato per una maschera. La stessa logica viene utilizzata in fase di valutazione. Per ogni task, ROC-AUC e average precision vengono calcolate considerando solamente le molecole per cui l'etichetta è disponibile, e i risultati vengono poi mediati tra i task. Se in uno split un task contiene una sola classe, la relativa AUC non è definita e quel task viene escluso dalla media anziché essere considerato come un risultato pari a zero.

3.3 Lo split scaffold

La scelta della partizione dei dati è stata una delle decisioni metodologiche più importanti del progetto, perché determina quanto il test rappresenti realmente la capacità del modello di generalizzare a molecole nuove. Con uno *split casuale*, le molecole vengono distribuite casualmente tra training e test. In questo modo, però, può capitare che molecole molto simili, o appartenenti alla stessa famiglia strutturale, finiscano in entrambi gli insiemi. Il modello potrebbe quindi ottenere buone prestazioni sfruttando somiglianze con molecole già osservate durante il

training, rendendo il risultato meno indicativo della sua capacità di generalizzare a strutture realmente nuove. Lo *scaffold split* affronta questo problema raggruppando le molecole in base al loro *scaffold*, cioè allo scheletro strutturale principale che caratterizza la molecola, e assegnando ogni gruppo interamente al training, validation oppure test. Di conseguenza, nel test sono presenti scheletri che non compaiono nel training: il modello deve quindi generalizzare a strutture molecolari che non ha incontrato durante l’addestramento. Questa scelta rende il problema più realistico, ma introduce una difficoltà ulteriore: training e test possono avere distribuzioni diverse. Nel dataset utilizzato, ad esempio, il tasso di molecole positive differisce mediamente di 3,1 punti percentuali tra i due insiemi, con uno scarto massimo di 10,4 punti nel task SR-ARE, dove i positivi sono il 14,7% nel training e il 25,1% nel test (Figura 2). Questo *distribution shift*, insieme alla maggiore difficoltà dello scaffold split, contribuisce a spiegare perché i risultati ottenuti in questo progetto siano inferiori a quelli riportati in alcuni lavori su Tox21 che utilizzano uno split casuale. Uno degli esperimenti successivi analizza proprio quanto la scelta dello split influisca sulle prestazioni del modello.

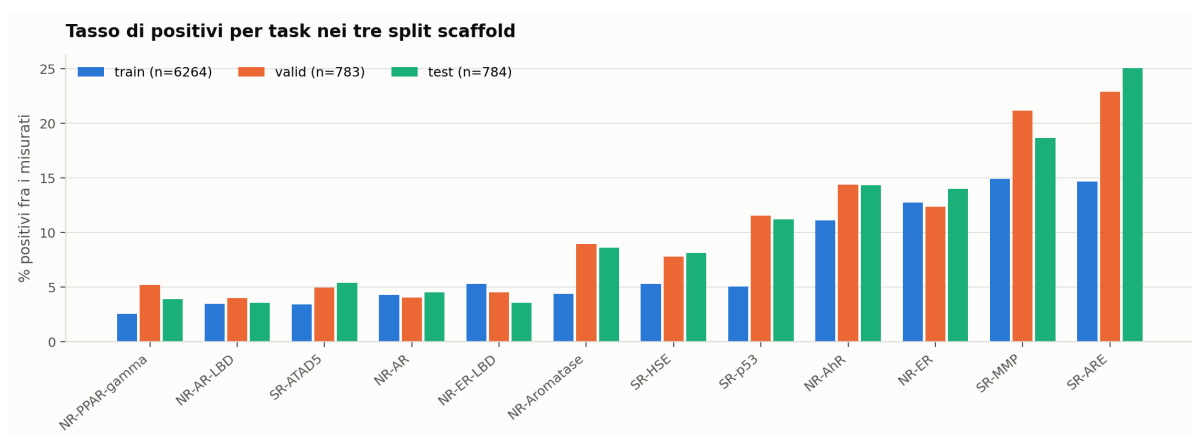


Figura 2: Tasso di positivi per task nei tre insiemi dello split scaffold. Le differenze fra train e test sono lo spostamento della distribuzione che il modello deve assorbire.

3.4 I dati per la baseline

La Random Forest richiede una rappresentazione vettoriale delle molecole, anziché un grafo. Per questo ho utilizzato gli SMILES dello stesso dataset OGB e li ho convertiti in fingerprint di Morgan (ECFP4), con raggio 2 e dimensione di 2048 bit, utilizzando RDKit. Il fingerprint trasforma questa rappresentazione in un vettore binario in cui ogni bit indica la presenza o l’assenza di una determinata sottostruttura locale. Per ogni atomo, vengono considerate le informazioni relative all’atomo stesso e a quelli raggiungibili entro due legami. Per garantire un confronto corretto con la GNN, ho utilizzato esattamente le stesse partizioni definite da OGB, leggendo gli split direttamente dai relativi file CSV. Di conseguenza, Random Forest e GNN vengono addestrate e valutate sulle stesse molecole nei tre insiemi. Una differenza strutturale fra i due approcci va però tenuta presente nel confronto: mentre la GNN è un unico modello multi-task, che condivide la stessa rappresentazione interna fra i 12 task, la baseline addestra una **foresta separata per ciascun task**, ognuna sulle sole molecole effettivamente misurate su quel task. Sono quindi 12 modelli indipendenti, ciascuno con 500 alberi e `class_weight="balanced"` per compensare lo sbilanciamento delle classi.

4 Modello principal

La **GIN** (*Graph Isomorphism Network*) è l'architettura principale utilizzata nel progetto. A ogni strato, la rappresentazione di un nodo viene aggiornata combinando la sua rappresentazione precedente con quelle dei nodi vicini:

$$h_v^{(k)} = \text{MLP}^{(k)} \left(\left(1 + \epsilon^{(k)} \right) h_v^{(k-1)} + \sum_{u \in \mathcal{N}(v)} h_u^{(k-1)} \right).$$

La GIN utilizza la *somma* per aggregare le informazioni dei vicini. A differenza della media o del massimo, la somma mantiene anche l'informazione relativa al numero di vicini: la media e il massimo non distinguono un nodo con un solo vicino da uno con cinque vicini identici, mentre la somma sì. Questa caratteristica rende la GIN particolarmente espressiva. Il parametro ϵ , appreso durante il training, permette inoltre di controllare separatamente il contributo della rappresentazione del nodo rispetto a quello dei suoi vicini. Per la configurazione di riferimento sono state adottate le impostazioni predefinite dell'implementazione OGB, ovvero: 5 strati con dimensione nascosta pari a 300, *batch normalization* e dropout pari a 0,5 dopo ogni strato, ReLU dopo ogni strato tranne l'ultimo, e *global mean pooling*. La rappresentazione ottenuta per ciascuna molecola viene infine passata ad una linear che produce 12 logit, uno per ciascun task.

4.1 Che cosa riceve concretamente la rete

Prima di descrivere l'architettura conviene fissare che cosa sia, in pratica, l'input del modello. La Figura 3 segue una molecola del dataset, la benzammide, dai suoi SMILES fino ai tensori che entrano nella rete.

Una molecola è quindi descritta da due soli tensori: la matrice $\mathbf{x}$ di dimensione $[n_{\text{atomi}} \times 9]$, che contiene i nove attributi categorici di ogni atomo, e la matrice `edge_index` di dimensione $[2 \times 2m]$, che elenca i legami come coppie di indici. I legami compaiono in entrambi i versi perché il grafo è non orientato e il message passing deve poter percorrere ogni legame nelle due direzioni: la benzammide ha 9 legami e quindi 18 colonne in `edge_index`. I valori di $\mathbf{x}$ sono indici e non quantità, in quanto identificano una categoria. È esattamente per questo che servono gli *embedding* descritti nella Sezione 3.1 e non si può passare $\mathbf{x}$ direttamente a uno strato lineare.

4.2 L'architettura implementata

Il modello di riferimento è composto da quattro blocchi. Indicando con n il numero di atomi della molecola:

1. **AtomEncoder.** Ciascuno dei 9 attributi categorici ha una propria tabella di embedding di dimensione 300, per un totale di 174 righe. I nove vettori di un atomo vengono *sommati*, e producono la rappresentazione iniziale $h^{(0)}$, una matrice $n \times 300$.
2. **Cinque strati di message passing.** Ogni strato applica nell'ordine: GINConv, *batch normalization*, ReLU e dropout. La ReLU è l'unica operazione che viene omessa nell'ultimo strato, in modo che la rappresentazione finale dei nodi possa assumere anche valori negativi; la *batch normalization* e il dropout sono invece applicati anche lì. Il dropout con probabilità 0,5 è l'unica forma di regolarizzazione usata.

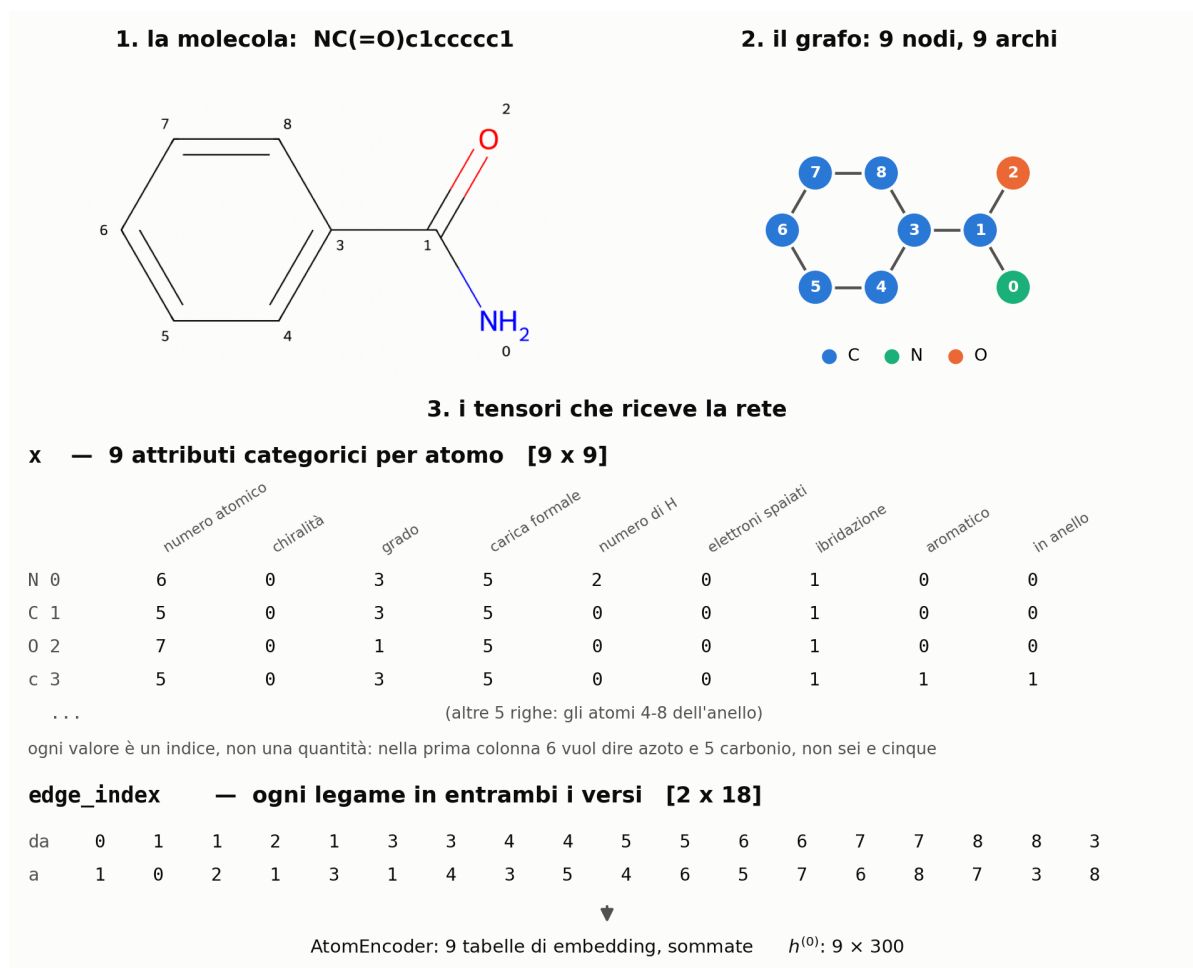


Figura 3: Come una molecola del dataset diventa l'input della GIN. In alto la molecola con gli atomi numerati e il grafo corrispondente; in basso i due tensori che la rete riceve davvero: la matrice **x** degli attributi degli atomi e la lista **edge_index** dei legami. Gli attributi sono *indici* di categorie, non grandezze numeriche, e vengono trasformati in vettori dall'**AtomEncoder**.

- Global pooling.** Le n rappresentazioni dei nodi vengono ridotte a un solo vettore di 300 componenti per molecola. Nella configurazione di riferimento è la media; *sum* e *max* sono le due alternative confrontate in E4.
- Strato lineare.** Vi è un unico strato lineare $300 \rightarrow 12$ produce i 12 logit, uno per task. La rete restituisce i logit e non le probabilità: la sigmoide è applicata dentro la loss durante l'addestramento e prima del calcolo delle metriche in fase di valutazione.

L'MLP che compare nella formula della GIN non è un singolo strato lineare, ma una piccola rete a due strati, composta da $300 \rightarrow 600$, batch normalization, ReLU e $600 \rightarrow 300$. È qui che si concentra quasi tutta la capacità del modello. Il parametro ϵ è lasciato addestrabile (**train_eps=True**), permettendo alla rete di regolare autonomamente quanto pesare la rappresentazione del nodo rispetto alla somma dei vicini, anziché mantenerla fissa a zero. Un dettaglio importante è che tutti e cinque gli strati sono *distinti*: i pesi non sono condivisi fra le iterazioni di message passing, quindi ogni strato impara una trasformazione diversa.

4.3 Le varianti di confronto

Per valutare l'importanza della scelta del tipo di convoluzione, ho confrontato la GIN con altre due architetture, mantenendo invariata la struttura del modello e modificando solo il livello di message passing. In particolare, ho utilizzato la **GCN**, che normalizza il contributo dei vicini in funzione del loro grado e ha una capacità espressiva teoricamente inferiore rispetto alla GIN, e la **GINE**, un'estensione della GIN che incorpora anche le caratteristiche degli archi nel message passing. In questo modo il confronto permette di valutare l'effetto della convoluzione mantenendo costanti le altre scelte architetturali.

5 Impostazione sperimentale

Ogni *run* viene addestrata utilizzando l'ottimizzatore Adam, con *learning rate* pari a 10^{-3} e batch di dimensione 32. Al termine di ogni epoca viene calcolata la ROC-AUC media sul validation set. I pesi corrispondenti alla migliore prestazione vengono salvati e utilizzati successivamente per la valutazione finale. L'addestramento viene interrotto anticipatamente se la ROC-AUC non migliora per 20 epoche consecutive. È previsto anche un limite massimo di epoche, ma non è mai stato raggiunto: in tutte le run l'early stopping è intervenuto prima, dopo un numero di epoche compreso fra 27 e 140. Il test set viene utilizzato **una sola volta**, al termine dell'addestramento e dopo aver ricaricato i pesi migliori. Non viene quindi utilizzato per scegliere l'architettura, gli iperparametri o l'epoca migliore, evitando di introdurre informazioni del test nella fase di selezione del modello. Ogni configurazione viene eseguita con almeno 3 seed differenti e i risultati sono riportati come media e deviazione standard. Per tre configurazioni particolarmente rilevanti, il modello di riferimento, il *global max pooling*, che ha ottenuto le prestazioni migliori, e il *global sum pooling*, che ha mostrato la maggiore variabilità, sono state effettuate ulteriori ripetizioni, arrivando a 5–6 seed.

6 Esperimenti e risultati

Tutti gli esperimenti sono costruiti attorno a una **configurazione di riferimento**, descritta nella sezione precedente. Ogni esperimento cambia **una sola cosa** rispetto a questo riferimento e lasciando invariato il resto.

6.1 Che cosa ho voluto misurare

Sono stati definiti sette esperimenti in modo da analizzare sistematicamente l'effetto delle principali scelte progettuali sul risultato finale. Nel loro insieme, gli esperimenti considerano tre distinti livelli di analisi: il *modello* (E1–E3), la modalità con cui il modello rappresenta e sintetizza la molecola, nonché la sua capacità (E4–E5), e infine il *protocollo sperimentale*, che comprende il partizionamento dei dati e la definizione della loss (E6–E7). La Tabella 2 riassume il piano sperimentale adottato.

Le metriche utilizzate per valutare gli esperimenti sono la **ROC-AUC** e l'**average precision**. La **ROC-AUC** misura quanto bene il modello riesca a distinguere le molecole positive da quelle negative considerando l'intero ordinamento dei punteggi. In particolare, può essere interpretata come la probabilità che, scegliendo casualmente una molecola positiva e una negativa, il modello assegni un punteggio maggiore alla prima. È inoltre relativamente poco influenzata dallo sbilanciamento tra le classi. L'**average precision**, invece, si concentra maggiormente sulle molecole a cui il modello assegna i punteggi più alti, valutando quanto tra queste siano

Tabella 2: Il piano degli esperimenti.

	Cosa cambia rispetto al riferimento	Obiettivo dell'esperimento
E1	Modello non neurale: ECFP4 + Random Forest	Valutare il vantaggio dell'apprendimento della rappresentazione dal grafo rispetto a una rappresentazione costruita manualmente.
E2	GCN al posto di GIN	Valutare l'effettivo beneficio della maggiore espressività teorica della GIN rispetto alla GCN.
E3	GINE al posto di GIN	Valutare il contributo informativo delle caratteristiche dei legami rispetto alle sole caratteristiche atomiche.
E4	Pooling <i>sum</i> e <i>max</i> al posto di <i>mean</i>	Analizzare l'influenza della strategia di pooling sulla rappresentazione complessiva della molecola.
E5	2 o 3 strati al posto di 5	Valutare l'impatto della capacità del modello e dell'ampiezza del raggio di message passing sulle prestazioni.
E6	Split casuale al posto dello scaffold	Valutare l'influenza della strategia di partizionamento dei dati sulle prestazioni, a parità di modello.
E7	<code>pos_weight</code> nella BCE	Valutare l'effetto della compensazione esplicita dello sbilanciamento tra le classi, con particolare attenzione all'identificazione dei campioni positivi.

effettivamente positive. Per questo motivo è particolarmente informativa in presenza di classi fortemente sbilanciate.

6.2 Il quadro complessivo

Tabella 3: Risultati di tutti gli esperimenti. n è il numero di seed, il $\pm$ è la deviazione standard fra seed. La selezione del modello avviene sulla ROC-AUC di validation; AP è l'average precision.

	configurazione	n	AUC valid	AUC test	AP test
E1	fingerprint ECFP4 + Random Forest	3	0,7796 $\pm$ 0,0007	0,7325 $\pm$ 0,0010	0,3939 $\pm$ 0,0018
–	GIN 5 strati, mean (riferimento)	5	0,7772 $\pm$ 0,0035	0,7432 $\pm$ 0,0031	0,3453 $\pm$ 0,0037
E2	GCN al posto di GIN	3	0,7794 $\pm$ 0,0031	0,7340 $\pm$ 0,0015	0,3594 $\pm$ 0,0077
E3	GINE, con feature dei legami	3	0,7740 $\pm$ 0,0045	0,7360 $\pm$ 0,0118	0,3334 $\pm$ 0,0279
E4	pooling sum	6	0,7554 $\pm$ 0,0209	0,7142 $\pm$ 0,0159	0,3111 $\pm$ 0,0295
E4	pooling max	5	0,8051 $\pm$ 0,0021	0,7564 $\pm$ 0,0091	0,3632 $\pm$ 0,0085
E5	2 strati	3	0,7827 $\pm$ 0,0023	0,7422 $\pm$ 0,0003	0,3696 $\pm$ 0,0048
E5	3 strati	3	0,7788 $\pm$ 0,0090	0,7457 $\pm$ 0,0094	0,3529 $\pm$ 0,0210
E6	split casuale invece di scaffold	3	0,8372 $\pm$ 0,0076	0,8400 $\pm$ 0,0251	0,4792 $\pm$ 0,0371
E7	<code>pos_weight</code> nella loss	3	0,7666 $\pm$ 0,0060	0,7334 $\pm$ 0,0055	0,3052 $\pm$ 0,0294

Prima di analizzare i singoli esperimenti conviene guardare la tabella nel suo insieme, in quanto è interessante vedere **quanto pesano scelte di natura diversa**. Ordinando gli effetti per ampiezza si vedono tre scale ben distinte:

- **la partizione dei dati vale circa dieci punti di AUC** (E6: da 0,7432 con lo scaffold a 0,8400 con lo split casuale, a parità di tutto il resto);
- **il pooling vale circa quattro punti** (E4: da 0,7142 con *sum* a 0,7564 con *max*);
- **la convoluzione e la profondità valgono meno di un punto** (E2, E3 ed E5 stanno tutte fra 0,7340 e 0,7457).

Le scelte che inizialmente potevano sembrare più importanti, cioè il tipo di convoluzione e la profondità della rete, hanno avuto un impatto relativamente contenuto sulle prestazioni. Al contrario, la scelta del metodo di pooling e quella della strategia di partizione dei dati hanno

prodotto variazioni molto più significative. Questo rappresenta una delle principali conclusioni metodologiche emerse dagli esperimenti. Un ulteriore elemento interessante emerge dal confronto tra le due colonne della ROC-AUC. Con lo *scaffold split*, la ROC-AUC sul test risulta sistematicamente inferiore a quella sul validation set, con una differenza di circa 3–5 punti in tutte le configurazioni. Ad esempio, il modello di riferimento passa da 0,7772 sul validation a 0,7432 sul test, mentre il *max pooling* passa da 0,8051 a 0,7564. Con lo split casuale utilizzato nell'E6, invece, questa differenza scompare: la ROC-AUC è 0,8372 sul validation e 0,8400 sul test. Questo comportamento non indica un *overfitting* sul validation set in quanto quest'ultimo non viene utilizzato per aggiornare i pesi, ma solamente per selezionare l'epoca migliore. La differenza osservata con lo *scaffold split* è piuttosto dovuta al fatto che validation e test contengono gruppi di scheletri molecolari differenti e rappresentano quindi due insiemi distinti di strutture non viste durante il training. Con lo split casuale, invece, i tre insiemi sono estratti dalla stessa distribuzione e, di conseguenza, le prestazioni su validation e test risultano molto più simili.

6.3 I singoli esperimenti

E1: La baseline non è facile da battere. La Random Forest basata sulla rappresentazione ECFP4 ottiene una ROC-AUC di 0,7325 e un'average precision di 0,3939 sul test set. La GIN di riferimento raggiunge invece una ROC-AUC di 0,7432, circa un punto percentuale in più, ma ottiene un'average precision di 0,3453, quasi cinque punti percentuali in meno. Le due metriche portano quindi a risultati differenti: la GIN risulta migliore secondo la ROC-AUC, mentre la Random Forest ottiene una prestazione superiore secondo l'average precision. La differenza tra i due modelli è inoltre più marcata in termini di AP che di ROC-AUC. Per questo motivo nel seguito vengono riportate sempre entrambe le metriche. Il divario non nasce da un singolo task anomalo: confrontando i due modelli task per task, la GIN è davanti alla Random Forest su 8 task su 12 secondo la ROC-AUC, ma solo su 4 su 12 secondo l'average precision. Le due metriche premiano quindi davvero due comportamenti diversi. È inoltre importante sottolineare che la baseline non utilizza una configurazione semplificata del problema: la Random Forest impiega gli stessi split ufficiali del modello GIN, 500 alberi e `class_weight="balanced"`.

E2: L'espressività della convoluzione non è così vantaggiosa. La GCN raggiunge 0,7340 di AUC contro 0,7432 della GIN: circa nove decimi di punto a favore della GIN, quindi la direzione attesa dalla teoria. Sull'AP però l'ordine si inverte, con 0,3594 della GCN contro 0,3453 della GIN. Il punto interessante è che questo avviene con un utilizzo molto inferiore di parametri e con un tempo di addestramento inferiore. La GIN è dimostrabilmente più espressiva della GCN nel distinguere strutture di grafo diverse, ma qui questa capacità in più non si traduce in un vantaggio pratico: con lo scaffold split la difficoltà non è distinguere fra loro le molecole di training, è **trasferire ciò che si è imparato a scheletri molecolari mai visti**, e su quel fronte l'espressività non aiuta.

E3: Le feature dei legami non aggiungono informazione utile. La GINE, che inietta nel message passing anche gli attributi degli archi, ottiene 0,7360 di AUC e 0,3334 di AP: leggermente sotto la GIN su entrambe le metriche e con tempi di addestramento maggiori. La spiegazione più plausibile è che l'informazione dei legami sia in buona parte **ridondante**: il tipo di legame fra due atomi è largamente determinato da quali atomi sono, dalla loro ibridazione e dal loro vicinato, tutte informazioni che la rete riceve già attraverso gli attributi dei nodi. Fornirle esplicitamente aggiunge parametri e tempo.

E4: Il pooling è la scelta architetturale che conta davvero. Questo è l'esperimento con l'effetto più netto fra quelli sul modello: *max* arriva a 0,7564, *mean* a 0,7432 e *sum* a 0,7142. Fra la configurazione migliore e la peggiore ci sono più di quattro punti di AUC, cioè un divario quattro volte più grande di quello fra due convoluzioni diverse. Il vantaggio del *max* è ancora più evidente sul validation, dove raggiunge 0,8051: è il valore più alto fra tutte le configurazioni addestrate sullo split scaffold, e tutte e cinque le run del *max* stanno sopra tutte le run di ogni altra configurazione, senza eccezioni. La lettura che ne do è che i tre pooling implicano tre ipotesi diverse su *che cosa rende tossica una molecola*. Il *sum* somma i contributi di tutti gli atomi e produce quindi una rappresentazione la cui scala cresce con la dimensione della molecola: dato che qui gli atomi vanno da 1 a 132, la rete deve dedicare parte della propria capacità ad assorbire una variazione di scala che con la tossicità non ha nulla a che vedere. Il *mean* normalizza per la dimensione, ma proprio per questo diluisce: una sottostruttura pericolosa presente in un solo atomo pesa poco se la molecola ne ha cento. Il *max* prende, per ciascuna dimensione della rappresentazione, il valore più alto raggiunto in tutta la molecola, e si comporta quindi da **rilevatore di sottostrutture**: individua le sottostrutture che attivano più fortemente una data caratteristica, che può essere quella associata alla tossicità.

E5: la profondità non rappresenta il principale collo di bottiglia. L'aumento della profondità del modello non produce un miglioramento significativo delle prestazioni: 2 strati raggiungono 0,7422, 3 strati 0,7457 e 5 strati 0,7432. L'incremento sostanziale della capacità del modello non si traduce quindi in un beneficio misurabile. L'esperimento consente di escludere la profondità come principale limite del modello. Se le prestazioni fossero vincolate dalla capacità del modello o dalla dimensione del suo campo recettivo, l'aumento del numero di strati avrebbe dovuto produrre un miglioramento: con 5 strati, infatti, ogni atomo può integrare informazione proveniente da un vicinato più ampio, fino a una distanza di 5 hop, rispetto ai 2 hop della configurazione più piccola. Il fatto che questo incremento non porti a un vantaggio suggerisce che il limite delle prestazioni debba essere ricercato altrove. Il fattore più plausibile è la limitata disponibilità di esempi positivi: confrontando task per task la GIN di riferimento con la Random Forest, il vantaggio della rete sull'average precision cresce con il numero di molecole positive presenti nel training (correlazione $r = +0,50$), e le cinque perdite più gravi si concentrano tutte su task con meno di 300 positivi, fra cui NR-PPAR- γ , che ne ha soltanto 134. Un ulteriore elemento a sostegno di questa interpretazione è fornito dalla configurazione a 2 strati, che raggiunge un AP pari a 0,3696, il valore più elevato tra tutti i modelli neurali considerati. In presenza di poche centinaia di esempi positivi per task, una architettura più compatta può quindi risultare più efficace nel processo di generalizzazione, evitando di introdurre capacità aggiuntiva che i dati disponibili non sono sufficienti a sfruttare.

E6: Lo split scaffold costa dieci punti. Sostituendo la partizione scaffold con una casuale delle stesse dimensioni, e lasciando identico tutto il resto, l'AUC di test sale da 0,7432 a 0,8400 e l'AP da 0,3453 a 0,4792. Quasi dieci punti di AUC e quasi quattordici di AP di differenza **senza toccare il modello**: è più del doppio di tutto ciò che sono riuscito a guadagnare con le scelte architetturelle messe insieme. Il motivo è quello discusso nella Sezione 3.3: con la partizione casuale molecole della stessa famiglia strutturale finiscono sia in training sia in test, e il modello può cavarsela riconoscendo scheletri che ha già visto invece di generalizzare a chimica nuova. Da qui due conseguenze concrete. La prima, metodologica, è che i confronti con la letteratura vanno fatti a parità di split, perché un lavoro che riporta 0,84 su Tox21 non sta necessariamente descrivendo un modello migliore del mio. La seconda è che questo esperimento giustifica a posteriori la scelta di usare lo scaffold split come riferimento: se l'obiettivo dichiarato è selezionare composti nuovi da testare, la stima ottimista non è quella utile.

E7: Il riequilibrio delle classi nella loss peggiora le prestazioni. In questo esperimento, ogni esempio positivo viene pesato nella loss in proporzione allo sbilanciamento tra le classi, utilizzando il rapporto tra esempi negativi e positivi calcolato sul training set. I pesi risultano compresi tra 5,7 per SR-MMP e 38,6 per NR-PPAR- γ . Questa strategia porta a un peggioramento delle prestazioni: l'AUC scende a 0,7334, mentre l'AP raggiunge 0,3052, il valore più basso tra tutti gli esperimenti considerati. Il risultato può sembrare controintuitivo, soprattutto perché una strategia analoga risulta efficace nella baseline, dove la Random Forest utilizza `class_weight="balanced"`. La differenza è nel modo in cui il peso viene utilizzato dai due modelli. Nella Random Forest, il peso modifica il criterio con cui vengono valutati gli split degli alberi; nella GNN, invece, il peso viene applicato direttamente alla *loss* e, di conseguenza, amplifica il contributo dei relativi gradienti durante l'aggiornamento dei parametri. Questo effetto diventa particolarmente marcato nei task più sbilanciati. Per NR-PPAR- γ , ad esempio, il contributo di soli 134 esempi positivi viene moltiplicato per circa 38. Se questi esempi contengono un segnale rumoroso, la rete rischia di adattarsi eccessivamente ad essi, producendo probabilità più elevate anche per esempi che non dovrebbero essere classificati come positivi. Il risultato è una peggiore calibrazione delle probabilità e, soprattutto, una classifica meno efficace nella regione in cui si concentrano le predizioni positive. Il risultato suggerisce quindi che, in questo contesto, il principale limite non sia il modo in cui vengono pesati i pochi esempi positivi, ma la loro disponibilità quantitativa: aumentare il loro peso nella loss non sostituisce la disponibilità di un numero sufficiente di esempi informativi.

7 Come è stato implementato il progetto

La struttura del repository è la seguente:

<code>src/data.py</code>	caricamento del dataset, split scaffold e casuale, statistiche
<code>src/models.py</code>	la classe GNN parametrica: GIN / GINE / GCN, tre pooling
<code>src/metrics.py</code>	loss mascherata, ROC-AUC e AP per task, <code>pos_weight</code>
<code>src/train.py</code>	un run completo, dalla CLI al file di risultati
<code>src/baselines.py</code>	fingerprint ECFP4 + Random Forest (E1)
<code>src/aggregate.py</code>	legge tutte le run e costruisce la tabella dei risultati
<code>src/plotstyle.py</code>	stile condiviso dalle figure, così notebook e script coincidono
<code>scripts/</code>	script bash per le ablation E2–E7 e lo script della Figura 3
<code>notebooks/</code>	analisi esplorativa e figure
<code>results/runs/</code>	un file JSON per run, con configurazione, storia e metriche

Per rendere il confronto tra gli esperimenti semplice e controllabile, `models.py` implementa un'unica classe parametrica, in cui convoluzione, numero di strati e strategia di pooling sono definiti tramite gli argomenti del costruttore. Gli esperimenti E2–E5 possono così essere configurati modificando un solo parametro alla volta da riga di comando, mantenendo invariato il resto della configurazione. La stessa impostazione è stata adottata per la gestione dei risultati. Ogni run viene salvato da `train.py` in un file JSON contenente la configurazione dell'esperimento, la storia dell'addestramento, la durata, il numero di parametri e le metriche finali per ciascun task. Successivamente, `aggregate.py` rilegge automaticamente i file, li raggruppa per configurazione e calcola media e deviazione standard delle metriche. Questa organizzazione rende la procedura riproducibile: i risultati non vengono trascritti manualmente e le tabelle possono essere rigenerate direttamente dai file prodotti durante gli esperimenti. Ogni valore riportato nella relazione è quindi tracciabile fino al relativo run.

8 Conclusioni

Nel complesso, i risultati ottenuti permettono di trarre alcune conclusioni sia sulle prestazioni dei modelli sia sui principali fattori che ne influenzano l'efficacia. Sul piano dei risultati, la GIN con pooling *max* è il modello migliore tra quelli addestrati, con una ROC-AUC di test pari a 0,7564, rispetto a 0,7325 della baseline basata su fingerprint. Questo vantaggio, tuttavia, non si riflette nell'Average Precision, dove la Random Forest rimane superiore (0,3939 contro 0,3632). Ne consegue che, su un dataset di queste dimensioni e caratterizzato da un forte sbilanciamento, una GNN addestrata da zero non offre un vantaggio pratico netto rispetto a un metodo classico ben configurato. In un sistema destinato alla selezione di composti da sottoporre a test di laboratorio, dove la precisione nelle prime posizioni è particolarmente importante, scegliere quindi la baseline, eventualmente affiancandola alla GNN. Sul piano formativo, emergono tre considerazioni principali. La prima è che alcune scelte apparentemente secondarie possono avere un impatto maggiore di quelle architetturali: il pooling modifica le prestazioni di circa quattro punti e il criterio di partizione dei dati di circa dieci, mentre il tipo di convoluzione e la profondità della rete restano sotto il punto. La seconda è l'importanza di scegliere la metrica prima di interpretare i risultati: considerando esclusivamente la ROC-AUC, la GNN sembrerebbe chiaramente superiore, mentre l'Average Precision porta a una conclusione diversa. La terza è la necessità di distinguere tra un limite del modello e un limite dei dati. Gli esperimenti E5 ed E7 hanno permesso di escludere sperimentalmente due spiegazioni alternative, cioè la capacità del modello e il modo in cui i pochi positivi vengono pesati nella loss; il confronto per task, in cui il vantaggio della rete cresce con il numero di positivi disponibili, indica come spiegazione più probabile la scarsità di esempi positivi, più che l'architettura utilizzata. I principali sviluppi futuri dovrebbero quindi concentrarsi proprio su questo limite. Una prima direzione consiste nel *pre-addestramento auto-supervisionato* su un corpus molto più ampio di molecole non etichettate, così da permettere alla rete di apprendere rappresentazioni utili prima di essere adattata a Tox21. Una seconda, più semplice, è combinare le due rappresentazioni in un unico modello: il confronto per task mostra che GNN e fingerprint non sbagliano sugli stessi task, e hanno quindi informazione complementare da offrire.